

[Back to TOC](#)

23.1 PORT SCANNING

- See Section 21.1 of Lecture 21 for the mapping between the ports and many of the standard and non-standard services. As mentioned there, each service provided by a computer monitors a specific port for incoming connection requests. There are 65,535 different possible ports on a machine.
- The main goal of port scanning is to find out which ports are **open**, which are **closed**, and which are **filtered**.
- **Looking at your machine from the outside, a given port on your machine is open only if you are running a server program on the machine and the port is assigned to the server.** If you are not running any server programs, then, from the outside, no ports on your machine are open. This could be the case with a brand new digital device that is not meant to provide any services to the rest of the world. **But, even with a device that was “clean” originally, should you happen to click accidentally on an email attachment consisting of malware, you could inadvertently end up installing a small server program in your machine that the bad guys could use to do their bad deeds.**

- When we say a port is **filtered**, what we mean is that the packets passing through that port are subject to the filtering rules of a firewall.
- If a port on a remote host is **open** for incoming connection requests and you send it a SYN packet, the remote host will respond back with a SYN+ACK packet (see Lecture 16 for a discussion of this).
- If a port on a remote host is **closed** and your computer sends it a SYN packet, the remote host will respond back with a RST packet (see Lecture 16 for a discussion of this).
- Let's say a port on a remote host is **filtered** with something like an **iptables** based packet filter (see Lecture 18) and your scanner sends it a SYN packet or an ICMP ping packet, you may not get back anything at all.
- **A frequent goal of port scanning is to find out if a remote host is providing a service that is vulnerable to buffer overflow attack (see Lecture 21 for this attack).**
- Port scanning may involve all of the 65,535 ports or only the ports that are well-known to provide services vulnerable to different security-related exploits.

[Back to TOC](#)

23.1.1 Port Scanning with Calls to `connect()`

- The simplest type of a scan is made with a call to `connect()`. The manpage for this system call on Unix/Linux systems has the following prototype for this function:

```
#include <sys/socket.h>

int connect(int sockfd, const struct sockaddr *address, socklen_t address_len);
```

where the parameter `sockfd` is the file descriptor associated with the internet socket constructed by the client (with a call to three-argument `socket()`), the pointer parameter `address` that points to a `sockaddr` structure that contains the IP address of the remote server, and the parameter `address_len` that specifies the length of the structure pointed to by the second argument.

- A call to `connect()` if successful completes a three-way handshake (that was described in Lecture 16) for a TCP connection with a server. The header file `sys/socket.h` includes a number of definitions of the [structs](#) needed for socket programming in C.

- When **connect()** is successful, it returns the integer 0, otherwise it returns -1.
- In a typical use of **connect()** for port scanning, if the connection succeeds, the port scanner immediately closes the connection (having ascertained that the port is open).

[Back to TOC](#)

23.1.2 Port Scanning with TCP SYN Packets

- Scanning remote hosts with SYN packets is probably the most popular form of port scanning.
- As discussed at length in Lecture 16 when we talked about SYN flooding for DoS attacks, if your machine wants to open a TCP connection with another machine, your machine sends the remote machine a SYN packet. If the remote machine wants to respond positively to the connection request, it responds back with a SYN+ACK packet, that must then be acknowledged by your machine with an ACK packet.
- In a port scan based on SYN packets, the scanner machine sends out SYN packets to the different ports of a remote machine. When the scanner machine receives a SYN+ACK packet in return for a given port, the scanner can be sure that the port on the remote machine is open. It is the “duty” of a good port-scanner to immediately send back to the target machine an RST packet in response to a received SYN+ACK packet so that the half-open TCP circuit at the target is closed immediately.

- Ordinarily, when a target machine receives a SYN packet for a closed port, it sends back an RST packet back to the sender.
- Note that when a target machine is protected by a packet-level firewall, it is the firewall rules that decide what the machine's response will be to a received SYN packet.

[Back to TOC](#)

23.1.3 The nmap Port Scanner

- **nmap** stands for “network map”. This open-source scanner, developed by Fyodor (see <http://insecure.org/>), is one of the most popular port scanners for Unix/Linux machines. There is good documentation on the scanner under the “Reference Guide” button at <http://nmap.org/>.
- **nmap** is actually more than just a port scanner. In addition to listing the open ports on a network, **it also tries to construct an inventory of all the services running in a network. It also tries to detect as to which operating system is running on each machine, etc.**
- In addition to carrying out a TCP SYN scan, **nmap** can also carry out TCP **connect()** scans, UDP scans, ICMP scans, etc. [Regarding UDP scans, note that SYN is a TCP concept, so there is *no* such thing as a UDP SYN scan. In a UDP scan, if a UDP packet is sent to a port that is *not* open, the remote machine will respond with an ICMP port-unreachable message. So the absence of a returned message can be construed as a sign of an open UDP port. However, as you should know from Lecture 18, a packet filtering firewall at a remote machine may prevent the machine from responding with an ICMP error message even when a port is closed.]

- As listed in its manpage, **nmap** comes with a large number of options for carrying out different kinds of security scans of a network. In order to give the reader a taste of the possibilities incorporated in these options, here is a partial description of the entries for a few of the options:

-sP : This option, also known as the “ping scanning” option, is **for ascertaining as to which machines are up in a network**. Under this option, nmap sends out ICMP echo request packets to every IP address in a network. Hosts that respond are up. But this does not always work since many sites now block echo request packets. To get around this, nmap can also send a TCP ACK packet to (by default) port 80. If the remote machine responds with a RST back, then that machine is up. Another possibility is to send the remote machine a SYN packet and wait for an RST or a SYN/ACK. **For root users, nmap uses both the ICMP and ACK techniques in parallel. For non-root users, only the TCP connect() method is used.**

-sS : The “-sS” option carries out a SYN scan.

-sT : The “-sT” option carries out a TCP `connect()` scan. See Section 23.1.1 for port scanning with calls to `connect()`.

-sU : This option sends a dataless UDP header to every port. As mentioned earlier in this section, the state of the port is inferred from the ICMP response packet (if there is such a response at all).

-sV : This is also referred to as “Version Detection”. After nmap figures out which TCP and/or UDP ports are open, it next tries to figure out what service is actually running at each of those ports. A file called `nmap-services-probes` is used to determine the best probes for detecting various services. In addition to determine the service protocol (http, ftp, ssh, telnet, etc.), nmap also tries to determine the application name (such as Apache httpd, ISC bind, Solaris telnetd, etc.), version number, etc.

- If nmap is compiled with OpenSSL support, it will connect to SSL servers to figure out the service listening behind the encryption.
- To carry out a port scan of your own machine, you could try (called as root)

```
nmap -sS localhost
```

where, as mentioned, the “-sS” option carries out a SYN scan. If you wanted to carry out an “aggressive” SYN scan of, say, moonshine.ecn.purdue.edu, you would call as root:

```
nmap -sS -A moonshine.ecn.purdue.edu
```

where you can think of the “-A” option as standing for either “aggressive” or “advanced.” This option enables OS detection, version scanning, script scanning, and more. [IMPORTANT: If the target machine has the DenyHosts shield running to ward off the dictionary attacks (See Lecture 24 for what that means) and you repeatedly scan that machine with the ‘-A’ option turned on, your IP address may become quarantined on the target machine (assuming that port 22 is included in the range of the ports scanned). When that happens, you will not be able to SSH into the target machine. The reason I mention this is because, when first using nmap, most folks start by scanning the machines they normally use for everyday work. Should the IP address of your machine become inadvertently quarantined in an otherwise useful-to-you target machine, you will have to ask the administrator of the target machine to restore your SSH privileges there. This would normally require deleting your IP address from six different files that are maintained by DenyHosts.]

- You can limit the range of ports to scan with the “-p” option, as in the following call which will cause only the first 1024 ports to be scanned:

```
nmap -p 1-1024 -sT moonshine.ecn.purdue.edu
```

- The larger the number of router/gateway boundaries that need to be crossed, the less reliable the results returned by **nmap**. As an illustration, I rarely get accurate results with **nmap** when I am port scanning a Purdue machine from home. [When scanning a remote machine several hops away, I sometimes get better results with my very simple port scanner `port_scan.pl` shown in Lecture 16. But, obviously, that scanner comes nowhere close to matching the amazing capabilities of **nmap**.]
- When I invoked **nmap** on localhost, I got the following result

```
Starting nmap 3.70 ( http://www.insecure.org/nmap/ ) at 2007-03-14 10:20 EDT
Interesting ports on localhost.localdomain (127.0.0.1):
(The 1648 ports scanned but not shown below are in state: closed)
PORT      STATE SERVICE
22/tcp    open  ssh
25/tcp    open  smtp
53/tcp    open  domain
80/tcp    open  http
111/tcp   open  rpcbind
465/tcp   open  smtps
587/tcp   open  submission
631/tcp   open  ipp
814/tcp   open  unknown
953/tcp   open  rndc
1241/tcp  open  nessus
3306/tcp  open  mysql
```

```
Nmap run completed -- 1 IP address (1 host up) scanned in 0.381 seconds
```

- By default, **nmap** first pings a remote host in a network before scanning the host. The idea is that if the machine is down, why waste time by scanning all its ports. But since many sites now block/filter the ping echo request packets, this strategy may bypass machines that may otherwise be up in a network. To change this behavior, the following sort of a call to **nmap** may produce richer results (at the cost of slowing down a scan):

```
nmap -sS -A -P0 moonshine.ecn.purdue.edu
```

The '-P0' option (the second letter is 'zero') tells **nmap** to **not** use ping in order to decide whether a machine is up.

- **nmap** can make a good guess of the OS running on the target machine by using what's known as "**TCP/IP stack fingerprinting**." It sends out a series of TCP and UDP packets to the target machine and examines the content of the returned packets for the values in the various header fields. These may include the sequence number field, the initial window size field, etc. Based on these values, **nmap** then constructs an OS "signature" of the target machine and sends it to a database of such signatures to make a guess about the OS running on the target machine.

[Back to TOC](#)

23.2 VULNERABILITY SCANNING

- The terms *security scanner*, *vulnerability scanner*, and *security vulnerability scanner* all mean roughly the same thing. Any such “system” may also be called just a *scanner* in the context of network security. Vulnerability scanners frequently include port scanning.
- A vulnerability scanner scans a specified set of ports on a remote host and tries to test the service offered at each port for its known vulnerabilities.
- **Be forewarned that an aggressive vulnerability scan may crash the machine you are testing.** It is a scanner’s job to connect to all possible services on all the open ports on a host. By the very nature of such a scan, a scanner will connect with the ports and test them out in quick succession. If the TCP engine on the machine is poorly written, the machine may get overwhelmed by the network demands created by the scanner and could simply crash. **That’s why many sysadmins carry out security scans of their networks no more than once a month or even once a quarter.**

[Back to TOC](#)

23.2.1 The Nessus Vulnerability Scanner

- According to the very useful web site “Top 125 Network Security Tools” (<http://sectools.org>), the source code for Nessus, which started out as an open-source project, was closed in 2005. Now for commercial applications you have to maintain a paid subscription to the company Tenable Computer Networks for the latest vulnerability signatures. However, it is still free for personal and non-commercial use. [The <http://sectools.org> website is a very useful place to visit to get an overview of the most commonly used computer security tools today. This website is maintained by the same folks who bring you the [nmap](#) scanner.]
- Nessus is a *remote security scanner*, meaning that it is typically run on one machine to scan all the services offered by a **remote** machine in order to determine whether the latter is safeguarded against all known security exploits.
- According to the information posted at <http://www.nessus.org>: Nessus is the world’s most popular vulnerability scanner that is used in over 75,000 organizations world-wide.
- The “Nessus” Project was started by Renaud Deraison in 1998. In 2002, Renaud co-founded Tenable Network Security with

Ron Gula, creator of the Dragon Intrusion Detection System and Jack Huffard. Tenable Network Security is the owner, sole developer and licensor for the Nessus system.

- The Nessus vulnerability scanning system consists of a server and a client. They can reside in two separate machines.
- The server program is called **nessusd**. This is the program that “attacks” other machines in a network. In a standard install of this software, the server is typically at the path `/opt/nessus/sbin/nessusd`.
- The client program is called **nessus**. The client, at the path `/opt/nessus/bin/nessus`, orchestrates the server, meaning that it tells the server as to what forms of attacks to launch and where to deposit the collected security information. The client packages different attack scenarios under different names so that you can use the same attack scenario on different machines or different attack scenarios on the same machine.
- While the server **nessusd** runs on a Unix/Linux machine, it is capable of carrying out a vulnerability scan of machines running other operating systems.
- The security tests for the Nessus system are written in a special

scripting language called **Network Attack Scripting Language (NASL)**. Supposedly, NASL makes it easy to create new security tests.

- Each security test, written in NASL, consists of an **external plugin**. As of April 2023, there are currently over 180,000 plugins available. New plugins are created as new security vulnerabilities are discovered. The command **nessus-update-plugins** can automatically update the database of plugins on your computer and do so on a regular basis. The following website provides up-to-date information regarding the available plugins:

<https://www.tenable.com/plugins>

- The client tells the server as to what category of plugins to use for the scan.
- **Nessus can detect services even when they are running on ports other than the standard ports.** That is, if the HTTP service is running at a port other than 80 or TELNET is running on a port other than port 23, Nessus can detect that fact and apply the applicable tests at those ports.
- Nessus has the ability to test SSLized services such as HTTPS, SMTPS, IMAPS, etc.

[Back to TOC](#)

23.2.2 Installing Nessus

- I went through the following steps to install this tool on my Ubuntu laptop:

- I downloaded the debian package from the Nessus website and installed it in my laptop with the following command:

```
dpkg -i Nessus-5.0.0-ubuntu1010_amd64.deb
```

When the package is installed, it displays the following message

All plugins loaded:

- You can start nessusd by typing `/etc/init.d/nessusd start`
- Then go to `https://pixie:8834/` to configure your scanner

where “pixie” is the name of my laptop. Installation of the package will deposit all the Nessus related software in the various subdirectories of the `/opt/nessus/` directory. In particular, all the client commands are placed in the `bin` subdirectory and all the root-required commands in the `sbin` directory. What that implies is that you must include `/opt/nessus/bin/` in the pathname for your account and `/opt/nessus/sbin/` in the pathname for the root account. You must also include `/opt/nessus/man/` in your `MANPATH` to access the documentation pages for Nessus.

- As root, you can now fire up the `nessusd` server by executing:

```
/etc/init.d/nessusd start
```

You can see that the Nessus server is up and running by doing any of the following:

```
netstat -n | grep tcp
netstat -tap | grep LISTEN
netstat -pltn | grep 8834
```

Any of these commands will show you that the Nessus server is running and monitoring port 8834 for scan requests from Nessus clients.

- Now, in accordance with the message you saw when you installed the debian package, point your web browser to <https://pixie:8834/> (with “pixie” replaced by the name you have given to your machine) to start up the web based wizard for installing the rest of the server software (mainly the plugins you need for the scans) through a feed from <http://support.tenable.com>. The web-based wizard will take you directly to this URL after you have indicated whether you want a home feed or a professional feed. Go for home feed for now — it’s free. I believe the professional feed could set you back by around \$1500 a year. When you register your server at the URL, you will receive a feed key that you must enter in the wizard for the installation to continue. If you are running a spam filter, make sure that it can accept email from `nessus.org`.

- After you have entered the feed key in the install wizard in your web browser, you will be asked for a username and a password in your role as a sysadmin for the Nessus server. (Note that this is comparable to a root privilege). Should you forget the password, you can re-create a new sysadmin password by executing the command
`‘/opt/nessus/sbin/nessus-chpasswd admin’` as root.
- After you you have entered the above info, the Nessus server will download all the plugins. As mentioned earlier, as of April 2023, there are over 180,000 of these plugins for all sorts of vulnerability scans. **Each plugin is based on a unique vulnerability signature.** Eventually, you will see a screen with the heading "Nessus Vulnerability Scanner". Under the header, you will see a bar that has "Listing Scans" on the left and a button for "New Scan" on the right. Click on the "New Scan" button to create a test scan to play with.
- If you wish to allow multiple clients (who may be on different hosts in a network) to run scan through your Nessus server, you can do that by executing the following command as root

nessus-adduser

For further information on this command, do ‘**man nessus-adduser**’. You can also remove users (clients) by executing as root the command ‘**nessus-rmuser**’.

- By the way, you can update your plugins by executing the command `'sudo ./nessus-update-plugins'` in the `/opt/nessus/sbin/` directory. This updating step only works if your server is registered with <http://www.nessus.org/register/>.
- You will find all the plugins in the following directory machine where the server is installed:

`/opt/nessus/lib/nessus/plugins/`

After you have updated the plugins, you can do `'ls -last | more'` in the above directory to see what sort of plugins were installed in the latest update.

[Back to TOC](#)

23.2.3 About the Nessus Client

- When you install the debian package as described in the previous subsection, the web-based install wizard I described there eventually takes you to a web based client. **Note that it is the client's job to tell the server what sort of a vulnerability scan to run on which machines.**
- Nessus also gives you a command-line client in the `/opt/nessus/bin` directory. The name of the client is `nessus`. If you do `'man nessus'`, you will see examples of how to call the client on a targeted machine. The vulnerability scan carried out by the command-line client depend on the information you place in scan config file whose name carries the `.nessus` suffix.
- The basic parameters of how the `nessus` client interacts with a Nessus server are controlled by the automatically generated `.nessusrc` file that is placed in client user's home directory.